

KaSCPar – Karlsruhe SystemC Parser Suite

What is KaSCPar

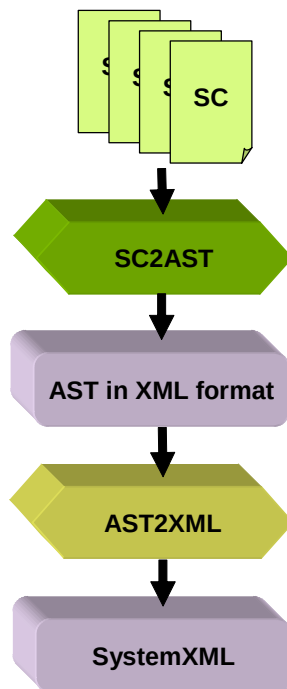
KaSCPar stands for Karlsruhe SystemC Parser Suite. It is a SystemC parser supporting the latest language version with dedicated tokens for SystemC 2.1 "keywords". The suite consists of two components for generating either the AST or an elaborated description of the SystemC design, both in XML format for easy usage.

Features

- supports C++ and SystemC 2.1
- based on C++ grammar with dedicated tokens for SystemC "keywords"
- uses GNU gcc preprocessor
- supports parsing of multiple sources files
- created by JavaCC & JTree
- generates two output files: the AST and the elaborated SystemC design in XML format

Description

The Karlsruhe SystemC Parser contains two phases. The first phase generates the AST and the second phase constructs the elaborated description of the SystemC design. The tool SC2AST generates only the AST and the tool SC2XML generates both the AST and the elaborated description of the SystemC design.



KaSCPar requirements

1. **J2SE** 5.0 (<http://java.sun.com/j2se/1.5.0/>)
2. **gcc** (<http://gcc.gnu.org/>)

We have tested under the following environments:

- a) gcc 3.3.5 on Linux SuSE 9.3
 - b) gcc 3.3.3 on Solaris 8
 - c) gcc 3.3.1 under cygwin on Windows 2003 and Windows 2000
3. **Apache Xerxes** (<http://xml.apache.org/xerces2-j/>)
 4. **sh** command interpreter (shell)

Please keep in mind that the **sh** and **gcc** paths must be set correctly in the PATH environment variable, because the SC2AST uses these programs.

We recommend to copy the jar files in the \$JAVA_HOME/jre/lib/ext/ directory, if not you will have to specify the location of the Xerces libraries in the command line used for calling the parser.

SC2AST

Synopsis

```
SC2AST      [-help]
            [-f source_file1 source_fileN]
            [-o output_source_file]
            [-i user_header_file_with_forward_declarations]
```

-help Show the usage message of the tool and exist.

-f Used to specify all input files. The files will be parsed according to the specified order. The files have to be separated by us of white spaces.

-o Name of the output source cpp file. This file consist of all input source files specified by **-f** option. The file **output_source_file.ast.xml** being generated by SC2AST contains the XML representation of the abstract syntax tree (AST) of the SystemC design.

-i Specifies the name of the user-defined included header file. In this file the user declares all types from the standard/system header files used in the program. This is necessary because the SC2AST tool avoids a standard header processing. For instance, if the string data type is used in the source code, "**class string;**" should be added in user header file.

Description

The generated AST file contains a token for each C/C++ and SystemC 2.1 language element. Furthermore, the AST file contains some meta information like column number, line number and file name as a reference to the original SystemC file and are saved as attributes of each AST token. The comments of source files are also saved as special token comment in the AST. The

parser uses the pre-processor from GNU gcc just to expand user defined macros of the SystemC design. The SC2AST class of *de.fzi.sim.kascpar.sc2ast* package provides a necessary interface which is reused in other applications. See class documentation for more details (SC2AST/docs/sdk/de/fzi/sim/kascpar/sc2ast/SC2AST.html).

SC2AST installation procedure

Directory structure of the tool (inside the SC2AST folder)

- /bin – *.jar, *.class files
- /docs/sdk – classes documentation (index.html), a parser grammar in the BNF form (SC2AST_BNF.html)
- /docs/examples – the SystemC 2.1 examples for the tool demonstration
- /src – Java and JavaCC sources
- /tests – a script to start the tool demonstration

Installation

1. Extract the SC2AST.tar.gz to a working directory.
2. Go to the working directory/SC2AST.
3. Sets the correct path to javac as a value of the JAVAC_OP variable in the SC2AST/Makefile.
4. Call the make utility to show the options:
 - make install – to compile the SC2AST tool and create the sc2ast.jar (SC2AST/bin)
 - make examples – to run the examples (SC2AST/docs/examples)
 - make clean – to remove all binary files and result files from the SC2AST/docs/examples folder
 - make clean_examples – to remove result files from the SC2AST/docs/examples only

Running the tool

Please make sure that you comply with the requirements (gcc and Xerces). You may invoke the parser from the command line:

```
java -jar sc2ast.jar [-f source_file1,..., source_fileN] [-i user-header_file]
[-o output_source_file] [-help]
```

The sc2ast.jar generated with make install option has been saved into the SC2AST/bin folder. There is also the sc2ast.jar from FZI in the SC2AST/bin/original_jar folder.